



1. Problem Title & Link

Title: LeetCode 1971 – Find if Path Exists in Graph

Link: <https://leetcode.com/problems/find-if-path-exists-in-graph/>

2. Problem Statement (Short Summary)

You are given:

- n vertices labeled from 0 to $n-1$
- An undirected graph represented by edges
- Two nodes: source and destination

Determine whether there exists **any path** from source to destination.

Return:

- true if a path exists.
- false otherwise.

3. Examples (Input → Output)

Example 1

Input

$n = 3$

edges = $[[0,1],[1,2],[2,0]]$

source = 0

destination = 2

Output

true

Example 2

Input

$n = 6$

edges = $[[0,1],[0,2],[3,5],[5,4],[4,3]]$

source = 0

destination = 5

Output



false

4. Constraints

- $1 \leq n \leq 2 \times 10^5$
- $0 \leq \text{edges.length} \leq 2 \times 10^5$
- Undirected graph
- No duplicate edges
- No self-loops

5. Core Concept (Pattern / Topic)

- Graph Traversal
- DFS
- BFS
- Connected Components

6. Thought Process (Step-by-Step Explanation)

What is the problem asking?

It does **not** ask for

- shortest path
- number of paths
- actual path

It only asks

Can we reach destination?

This immediately suggests

Graph Traversal

Step 1: Build the Graph

The graph is given as an edge list.

Convert it into an adjacency list.

Example

edges

0-1

1-2

2-3

Adjacency List

$0 \rightarrow [1]$



1 → [0,2]

2 → [1,3]

3 → [2]

Now we can easily visit neighbors.

Step 2: Start Traversal

Start from

source

Visit every reachable node.

Whenever

current == destination

return

True

Step 3: Avoid Infinite Loops

Graphs may contain cycles.

Example

0

/ \

1--2

Without a visited set,

0

↓

1

↓

2

↓



0

↓

1

↓

...

Infinite loop.

So maintain

visited

Which Traversal?

Both work.

- DFS
- BFS

Because we only need to know

Reachable?

Yes / No

Why This Works

Graph traversal explores **every node connected to the source**.

If the destination is among them,

a path exists.

Otherwise,

it doesn't.

7. Visual / Intuition Diagram (ASCII)

Example

0 ----- 1 ----- 2

|

3 ----- 4

Source

0



```
Destination
4
DFS
0

↓

1

↓

3

↓

4 ✓
Found.
```

8. Pseudocode (Language Independent)

```
build adjacency list

create visited

DFS(source)

    if node == destination

        return true

    mark visited

    visit all neighbors

return false
```

9. Code Implementation

Python (DFS - Iterative)

```
from collections import defaultdict

class Solution:
    def validPath(self, n: int, edges: list[list[int]], source: int, destination: int) -> bool:

        graph = defaultdict(list)

        for u, v in edges:
            graph[u].append(v)
```



```
graph[v].append(u)

stack = [source]
visited = {source}

while stack:

    node = stack.pop()

    if node == destination:
        return True

    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.add(neighbor)
            stack.append(neighbor)

return False
```

Python (Recursive DFS)

```
from collections import defaultdict

class Solution:

    def validPath(self, n, edges, source, destination):

        graph = defaultdict(list)

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        visited = set()

        def dfs(node):

            if node == destination:
                return True

            visited.add(node)

            for nei in graph[node]:
                if nei not in visited:
                    if dfs(nei):
                        return True

            return False

        return dfs(source)
```



Python (BFS)

```
from collections import defaultdict, deque

class Solution:

    def validPath(self, n, edges, source, destination):

        graph = defaultdict(list)

        for u, v in edges:
            graph[u].append(v)
            graph[v].append(u)

        queue = deque([source])
        visited = {source}

        while queue:

            node = queue.popleft()

            if node == destination:
                return True

            for nei in graph[node]:
                if nei not in visited:
                    visited.add(nei)
                    queue.append(nei)

        return False
```

Java (DFS - Iterative)

```
import java.util.*;

class Solution {

    public boolean validPath(int n, int[][] edges, int source, int destination) {

        List<List<Integer>> graph = new ArrayList<>();

        for(int i = 0; i < n; i++)
            graph.add(new ArrayList<>());

        for(int[] edge : edges){
            graph.get(edge[0]).add(edge[1]);
            graph.get(edge[1]).add(edge[0]);
        }

    }
```



```
Stack<Integer> stack = new Stack<>();
boolean[] visited = new boolean[n];

stack.push(source);
visited[source] = true;

while(!stack.isEmpty()){

    int node = stack.pop();

    if(node == destination)
        return true;

    for(int nei : graph.get(node)){

        if(!visited[nei]){

            visited[nei] = true;
            stack.push(nei);
        }
    }
}

return false;
}
```

10. Time & Space Complexity

Time Complexity

$O(V + E)$

- Visit each vertex once.
- Traverse each edge once.

Space Complexity

$O(V + E)$

- Adjacency List
- Visited Set
- DFS/BFS Stack or Queue

11. Common Mistakes / Edge Cases

- Forgetting to build an undirected graph.

`graph[u].append(v)`

`graph[v].append(u)`



Both are required.

- Forgetting the visited set.

Causes infinite loops.

- Using recursion on very large graphs.

Python may hit recursion depth.

Prefer iterative DFS.

- Marking visited after popping.

Safer to mark while pushing.

12. Detailed Dry Run

Example

$n = 6$

edges =

0-1

0-2

3-5

5-4

4-3

source = 0

destination = 5

Adjacency

$0 \rightarrow 1,2$

$1 \rightarrow 0$

$2 \rightarrow 0$



3 → 5,4

4 → 5,3

5 → 3,4

DFS

Stack	Visited
0	{0}
1,2	{0,1,2}
2	{0,1,2}
Empty	{0,1,2}

Destination never found.

Return

False

13. Common Use Cases (Real-Life / Interview)

- Network connectivity.
- Road navigation.
- Social network friend connections.
- Computer network routing.
- Connected component detection.

14. Common Traps (Important!)

- Forgetting the graph is **undirected**.
- Missing the visited set.
- Confusing DFS with shortest path (DFS doesn't guarantee the shortest path).
- Using Dijkstra unnecessarily.

15. Builds To (Related LeetCode Problems)

- LC 841 – Keys and Rooms
- LC 547 – Number of Provinces
- LC 200 – Number of Islands
- LC 695 – Max Area of Island
- LC 994 – Rotting Oranges



- LC 207 – Course Schedule
- LC 684 – Redundant Connection

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
DFS (Recursive)	$O(V+E)$	$O(V)$	Simple, recursion depth risk
DFS (Iterative)	$O(V+E)$	$O(V)$	Preferred in Python
BFS	$O(V+E)$	$O(V)$	Equally good
Union-Find (DSU)	$O(E \alpha(V))$	$O(V)$	Excellent for connectivity queries

17. Why This Solution Works (Short Intuition)

A path exists if and only if the destination belongs to the same connected component as the source. DFS or BFS explores every node reachable from the source. If the destination is visited during this traversal, a path exists; otherwise, it does not.

18. Variations / Follow-Up Questions

1. Return the actual path instead of true/false.
2. Find the shortest path (BFS for unweighted graphs).
3. Answer multiple connectivity queries efficiently using Union-Find (DSU).
4. Count the number of connected components.
5. Extend the solution to directed graphs.
- 6.